

1. Project Overview

Project Name: CoDesign System **Purpose:** A full-stack web application that allows users to explore an issue, provide structured “Why” and “How” responses, and generate AI-processed insights for analysis and reporting.

1.1 System Architecture

```
graph TD; Frontend["Frontend (React)"] --> API["REST API (Spring Boot)"]; API --> DB["Database (JPA / Hibernate)"]; DB --> AI["AI Processing Module"];
```

1.2 Technology Stack

Frontend

- React (v19)
- React Router
- Framer Motion
- Tailwind CSS + TDesign UI
- Recharts / ApexCharts

Backend

- Spring Boot (Java)
- RESTful APIs
- JPA / Hibernate

Deployment

- Backend: Render
- Frontend: Static hosting / Docker (Nginx)
- Containerization: Docker + docker-compose

2. Getting Started

2.1 Run with Docker (Recommended)

```
docker-compose up --build
```

2.2 Run Backend

```
cd backend  
./mvnw spring-boot:run
```

Backend runs at:

```
http://localhost:8080
```

2.3 Run Frontend

```
cd frontend  
npm install  
npm start
```

Frontend runs at:

```
http://localhost:3000
```

2.4 Environment Variables

Example:

```
REACT_APP_API_BASE_URL=http://localhost:8080/api
```

Production:

```
REACT_APP_API_BASE_URL=https://codesign-project.onrender.com/api
```

3. Backend Architecture

```
Controller → Service → Repository → Database
```

3.1 Controllers

- IssueController
- WhyResponseController

- HowResponseController
- ProfileController
- AIDataProcessingController

4. API Documentation

4.1 Issue APIs

Method	Endpoint	Description
POST	/api/issues	Create issue
GET	/api/issues	Get all issues
GET	/api/issues/{id}	Get issue
GET	/api/share/{shareId}	Get issue by shareId
POST	/api/issues/{id}/edit	Update issue
DELETE	/api/issues/{id}	Delete issue

4.2 Why APIs

Method	Endpoint
POST	/api/why

4.3 How APIs

Method	Endpoint
POST	/api/how

4.4 Profile APIs

Method	Endpoint
GET	<code>/api/profile/{submissionId}</code>
POST	<code>/api/profile/{submissionId}</code>
GET	<code>/api/profile/tags</code>
POST	<code>/api/profile/tags/custom</code>

4.5 AI Processing

Method	Endpoint
GET	<code>/api/ai-data/{shareId}</code>

5. Data Model

5.1 Core Entities

Issue

- `issueId`
 - `shareId`
 - `issueContent`
 - `state` (ACTIVE / DISABLED)
 - `publishedAt`
-

WhyResponse

- `shareId`
 - `stance`
 - `answer1-5`
 - `createdAt`
-

HowResponse

- `shareId`
 - `answer1-5`
 - `createdAt`
-

UserProfile

- submissionId
 - name
 - selectedTags (Many-to-Many)
-

Tag

- label
 - category
 - color
 - isSystem
 - createdBy
-

5.2 AI Data Structure

```
{
  "shareId": "abc123",
  "issueContent": "...",
  "stance": "...",
  "why": [{ "questionIndex": 1, "answer": "..."}],
  "how": [{ "questionIndex": 1, "answer": "..."}],
  "mergedWhyText": "...",
  "mergedHowText": "..."
}
```

5.3 AI Processing Logic

- Fetch latest Why & How responses
 - Clean invalid answers (idk, empty, symbols)
 - Convert to QA pairs
 - Merge into full text
-

6. Frontend Architecture

6.1 Structure

```
pages/
  admin/
  user/
components/
context/
```

7. Admin Pages

Dashboard

Overview of issues, metrics, and charts.

Create New Issue

- Input issue content
- POST `/api/issues`
- Generates share link:

```
/share/{shareId}
```

Issue Report

- Displays issue details
 - Shows metrics & charts
 - Supports:
 - AI report generation
 - Raw data export
-

Why Report / How Report

- Displays aggregated response insights
 - Word cloud & summary cards
-

Profile Report

- Displays user profile/tag analytics
-

Engagement Report

- Displays participation trends
-

User Data Management

- View users

- Export CSV
- Send voucher/update emails
- Manage templates

8. User Pages

WelcomePage

Flow entry point:

```
/share/:shareId
```

- Fetch issue via `/api/share/{shareId}`
- Create submission
- Store data in context + localStorage

ProfilePage

- Load tags → `/api/profile/tags`
- Load profile → `/api/profile/{submissionId}`
- Save → `/api/profile/{submissionId}`

WhyPage

- Displays issue content
- Multi-step input (5 questions)
- Collects why responses

HowPage

- Multi-step improvement input
- Supports early exit ("I don't know")

ThankYouPage

- Collect email & preferences
- PATCH `/api/submissions/{id}/thanks`

LoginPage

- POST `/api/users/login`
 - Store user session
-

CreateAccountPage

- POST `/api/users/signup`
 - Link submission → account
-

9. User Flow

```
/share/:shareId
→ Welcome
→ Profile
→ why
→ How
→ Thank You
→ (optional) Login / Register
```

10. Data Flow

```
User opens share link
→ Issue fetched
→ Submission created
→ Profile saved
→ why answers saved
→ How answers saved
→ AI processing endpoint used
```

11. Deployment

- Backend hosted on Render
 - Frontend deployed via static hosting / Docker
 - API base URL must match environment
-

12. Known Issues & Notes

- Deployment version may differ from local (cache issues)
- Some report pages use mock data
- AI processing depends on cleaned input quality
- Profile saving may fail if backend config missing

13. Troubleshooting

500 Error on Profile Save

- Check backend logs
 - Verify request payload structure
 - Ensure database connectivity
-

Frontend showing old version

- Clear browser cache
 - Use hard reload
-

API not responding

- Check backend deployment status
- Verify base URL